

SMART INDIA HACKATHON 2026

Problem Statement 26008

Intelligent Monitoring and Prediction of Conveyor Belt Joint Rupture and Damages in Iron Ore Mining Industry

Project Report

Category: Hardware

Theme: Smart Automation

Organization: Ministry of Steel — NMDC

An integrated IoT, edge computing, machine learning and visual monitoring prototype for conveyor-belt condition monitoring.

September 2026

1. Abstract

Conveyor belts are critical components in iron-ore mining operations, where damage to belt joints, belt tracking problems, abnormal vibration, thermal conditions and mechanical stress can affect the continuity of material handling. The proposed system is an intelligent monitoring platform designed to collect multiple operating signals and present the condition of a conveyor through a centralized dashboard.

The prototype combines an ESP32-based edge device, conveyor hardware, an external motor driver, camera-based monitoring, sensor interfaces, an edge gateway, PostgreSQL storage, vibration anomaly detection and a transparent multimodal evidence-fusion layer. The system is designed to identify abnormal operating conditions and provide interpretable evidence rather than treating the ML model as a black box.

The physical prototype and demonstration were tested separately, while the repository also provides a software testbench/simulation path so that the monitoring workflow can be evaluated without the complete physical setup.

2. Problem Statement

The project addresses intelligent monitoring and prediction of conveyor belt joint rupture and belt damage in the iron-ore mining industry. Conveyor systems operate continuously and can be exposed to vibration, temperature variation, mechanical loading, belt misalignment and other abnormal operating conditions.

A monitoring system therefore needs to combine information from different sources instead of relying on a single sensor. The project focuses on bringing these signals together into one monitoring pipeline and presenting actionable condition evidence through a dashboard.

3. Proposed Solution

The proposed solution is a multimodal conveyor monitoring platform. Data from physical hardware or the software testbench is collected at the edge, buffered locally, forwarded to the backend and stored for analysis. The backend performs vibration anomaly inference and combines the result with contextual evidence such as temperature, speed, load, belt tracking and camera information.

Monitoring source	Purpose
Vibration	Detect abnormal mechanical vibration patterns
Temperature	Monitor thermal operating conditions
Speed / RPM	Observe conveyor operating speed
Load	Represent mechanical loading information
Belt tracking	Detect lateral belt deviation / mistracking
Camera	Provide visual evidence of belt condition

The resulting condition is presented on the dashboard together with evidence and telemetry provenance.

4. System Architecture

The system follows a layered architecture that separates physical sensing, edge communication, backend processing and visualization.

Layer	Components	Function
Hardware	Conveyor prototype, DC geared motor, L298N, ESP32	Physical operations and data acquisition
Edge Gateway	Python edge agent + SQLite	Receive telemetry, buffer locally, deduplicate and forward
Backend	FastAPI + SQLAlchemy + PostgreSQL	API, persistence and processing
ML / Fusion	Isolation Forest + rule-based multimodal fusion	Vibration anomaly detection and evidence combination
Dashboard	React + Vite	Condition monitoring and visualization

Data flow: ESP32 / testbench → Edge Gateway → SQLite buffer → FastAPI backend → PostgreSQL → ML/fusion → Dashboard.

5. Hardware Prototype

The physical prototype uses a DC geared motor to drive the conveyor mechanism. An L298N dual H-bridge motor driver is used between the external motor supply and the motor, while the ESP32 provides low-voltage control signals for enable, direction and PWM speed control.

Hardware	Role
ESP32	Embedded controller and telemetry interface
DC geared motor	Drives the conveyor prototype
L298N	Motor direction and PWM speed control
USB camera	Visual monitoring
Vibration sensor interface	Mechanical vibration acquisition / testbench path
Temperature sensor interface	Thermal monitoring / testbench path
Speed sensor interface	RPM / speed monitoring / testbench path
Load sensor interface	Load monitoring / testbench path
Tracking sensor interface	Belt alignment monitoring / testbench path

The audited physical setup verified the conveyor motor, L298N driver and USB camera. During the software audit, the other listed sensor streams were represented as simulated/testbench inputs; the dashboard and telemetry schema explicitly distinguish those inputs from physical hardware telemetry.

6. ESP32 & Motor Control

Function	GPIO
I2C SDA	GPIO 21
I2C SCL	GPIO 22
Speed / tachometer	GPIO 19
Load proxy	GPIO 34
Tracking proxy	GPIO 27
L298N ENA / PWM	GPIO 25

Function	GPIO
L298N IN1	GPIO 26
L298N IN2	GPIO 33

The motor supply is kept on the external 12 V motor rail through the L298N, while the ESP32 operates from its low-voltage supply. A common ground reference is required for the control signals.

7. Software Stack

Technology	Use in project
C++ / Arduino ESP32	Embedded firmware
PlatformIO	Firmware build and project management
Python	Gateway and backend tooling
FastAPI	Backend REST API
PostgreSQL	Central telemetry storage
SQLite	Edge store-and-forward buffer
React + Vite	Monitoring dashboard
OpenCV	Camera capture / visual evidence
scikit-learn / Isolation Forest	Vibration anomaly detection

8. Edge Gateway

The edge gateway sits between the embedded hardware and the central backend. Its purpose is to make the telemetry path more resilient to temporary network or backend interruptions.

Telemetry is placed into a local SQLite queue and forwarded to the backend. The gateway also supports deduplication and retry behavior. This allows the software architecture to demonstrate store-and-forward operation instead of requiring continuous connectivity for every packet.

9. Backend & Database

The FastAPI backend receives telemetry through the monitoring API, validates the data contract, persists records in PostgreSQL and invokes the processing pipeline. Database storage provides a durable history of telemetry and derived condition information.

The backend also keeps the distinction between physical and simulated telemetry so that derived results can be interpreted with the correct provenance.

10. Machine Learning & Multimodal Fusion

The vibration monitoring component uses a frozen Isolation Forest model, identified as IF-v0.3.1. The model operates on vibration features including RMS values, peak vibration and kurtosis.

Feature	Meaning
RMS X / Y / Z	Vibration energy on the measured axes
Peak Z	Peak vibration magnitude on the Z axis
Kurtosis Z	Statistical indicator of impulsive vibration behavior

The ML component provides an anomaly signal. A separate transparent rule-based multimodal fusion layer combines this with operating context and other evidence. The project deliberately does not present the prototype as a calibrated 100%-accurate failure predictor or an exact Remaining Useful Life system.

11. Camera Monitoring

A USB camera provides visual evidence from the conveyor prototype. The camera path is decoupled from the telemetry pipeline so that image capture and processing do not unnecessarily block sensor ingestion. The dashboard can associate camera evidence with monitoring events.

12. Monitoring Dashboard

The React dashboard provides an operator-oriented view of conveyor condition. It presents sensor information, condition state, evidence and source information in a single interface.

Dashboard capability	Description
Condition overview	High-level operating state
Sensor cards	Vibration, temperature, speed, load and tracking information
Evidence view	Reasons and contributing signals behind a condition
Camera evidence	Visual frame associated with monitoring
Source badges	Distinguishes REAL_HARDWARE and DEMO_SIMULATED data
Scenario/testbench support	Allows controlled demonstration of abnormal conditions

13. Prototype Demonstration

The project was demonstrated using a physical conveyor prototype with a geared DC motor and L298N motor driver. The physical setup was tested for motor operation and camera capture, while the broader software monitoring pipeline can also be exercised using its testbench/simulation path.

The demonstration therefore shows both sides of the solution: the physical mechanism that produces conveyor operating conditions and the software platform that collects, analyzes and visualizes monitoring information.

14. Testing & Results

Area	Result
ESP32 firmware compilation	Successful PlatformIO build
Backend tests	12 tests passed in reported audit
Gateway tests	7 tests passed in reported audit
Offline resilience	6/6 reported stages passed
Frontend production build	Successful
ML model integrity	IF-v0.3.1 hash matched documented hash
Camera capture	Live 640×480 capture verified
Telemetry API	201 Created verified in integration test

These results establish the software prototype and integration path. They should be interpreted as prototype validation rather than a certification of industrial safety performance.

15. Key Features

Feature	Benefit
Multimodal monitoring	Combines multiple indicators instead of relying on one signal
Edge buffering	Maintains telemetry during temporary connectivity problems
Anomaly detection	Highlights unusual vibration behavior
Evidence fusion	Provides a more interpretable condition view
Visual monitoring	Adds camera-based evidence

Feature	Benefit
Source transparency	Clearly identifies simulated vs physical telemetry
Dashboard	Centralized operator view

16. Limitations

The current project is a prototype and proof-of-concept rather than a production-certified mining safety system. The available audit evidence does not establish calibrated failure probabilities, exact RUL, or universal performance across every conveyor design and operating environment.

During the software audit, several sensor inputs were represented through simulated/testbench sources. Physical validation of additional sensor hardware, long-duration conveyor data collection, site-specific calibration and field trials remain necessary for production deployment.

17. Future Scope

Area	Future development
Physical sensing	Integrate and calibrate all production-grade sensors
Data collection	Collect long-duration real conveyor operating and failure/degradation histories
Model development	Validate models against real conveyor-specific datasets
Visual AI	Develop dedicated belt/splice damage detection models
Calibration	Calibrate condition thresholds and confidence estimates for the target conveyor
Industrial deployment	Add ruggedized edge hardware, communications and environmental protection
Operational integration	Integrate alerts with maintenance workflows and plant monitoring systems

18. Conclusion

The project presents an integrated approach to conveyor-belt condition monitoring by combining embedded telemetry, edge buffering, backend analytics, vibration anomaly detection, camera evidence and a web-based dashboard. The architecture is designed to make abnormal conditions visible through multiple sources of evidence rather than depending on a single measurement.

The prototype demonstrates the complete software monitoring workflow and a physical conveyor mechanism, while maintaining a clear distinction between verified physical hardware and simulated/testbench inputs. This provides a practical foundation for further sensor integration, real-world data collection, model calibration and eventual industrial validation.

19. Project Technology Summary

Category	Technologies
Embedded	ESP32, Arduino, PlatformIO
Motor control	L298N, PWM, DC geared motor
Edge	Python, SQLite, telemetry buffering
Backend	FastAPI, SQLAlchemy, PostgreSQL
AI/ML	Isolation Forest, vibration feature analysis
Vision	OpenCV, USB camera
Frontend	React, Vite
Deployment / Development	Linux, Docker-based PostgreSQL environment, Git